26

# Who Is #include-ing You?

## Measuring the Impact of C++ Libraries

JOHN PARENT · Kitware

CppCon
THE C++ CONFERENCE
2026
September 12 – 18

BEFORE WE START: TWO TALKS WORTH YOUR TIME

# Go see my Kitware colleagues




This morning, time machine not included




Friday 9am

# How many of you can name the dependencies of a codebase you work on?



Hint: your build system does

# Who can name every project that depends on you?



THE QUESTION BEHIND THE TALK

# You (or some tool) can list your dependencies easily.

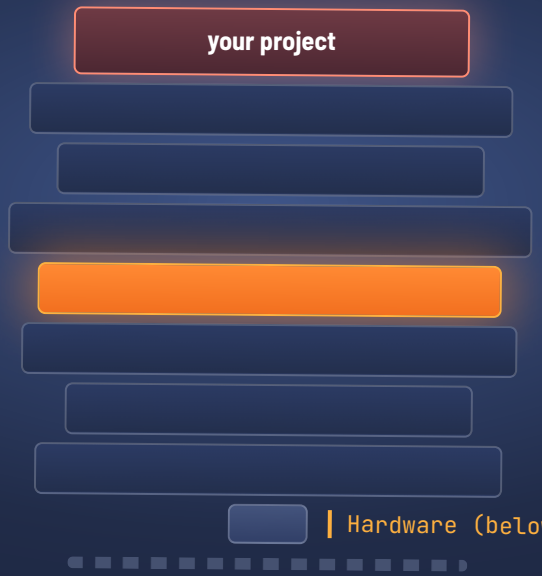
Nothing tells you what depends on you.

1

MOTIVATION

# Why should you care?

# Everything is a stack of dependencies

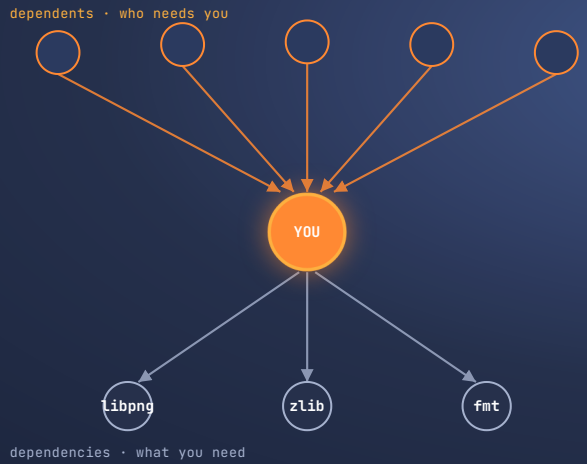



Every one of us is someone's dependent, all the way down the stack. Measuring who sits above **you** is the whole point.

# Your code (usually) is a cut-vertex

THE WHOLE PICTURE

**You are the knot in the bowtie**




Fan-in of dependents, fan-out of dependencies. This talk computes the top half.

**As a source**, you point **down** at what you consume. That graph is handed to you: a manifest, a `find_package`, a submodule, etc.

**As a sink**, an unknown number of projects point **at you**. Nobody hands you that graph.

How do we find that information?

Fuse both halves and you are the **knot in a bowtie**: fan-in above, fan-out below. This talk computes the top half.

# The most information we can get in one place, easily, right now

```
$ spack dependents zlib
==> Dependents of zlib
  boost   curl   git   hdf5
  libpng  mesa   python  ...
```

Conan, vcpkg, apt, Homebrew, and friends all answer the same reverse query. Spack is just the example.

> Real, precise, and **resolver-aware** - a package manager knows the exact build graph it resolves.

**But each only sees its own recipes.**

- Opt-in: a project exists here only if **someone wrote a recipe** for that manager.
- Most C++ software in the world is **never packaged**, in any of them.
- The managers **disagree**, rename (`py-`...), and split packages differently.
- Any one of them is a shallow, distorted slice of real usage.

# Doesn't this already exist?

Plenty of tools answer "who depends on this?" - for **other** ecosystems.

| Tool | What it maps | Why it's blind to C++ |
|---|---|---|
| **deps.dev** (Google) | npm · Cargo · PyPI · Go · Maven · NuGet | No C/C++ – there's **no registry to index** |
| GitHub dependency graph / **"Used by"** | manifests & lockfiles (`go.mod`, `Cargo.lock`…) | C++ ships **no manifest it parses**; "Used by" sits empty |
| Libraries.io · Ecosyste.ms | 30+ package registries | C++ = only the opt-in **Spack/Conan/vcpkg** slice, renamed & split |
| `spack/conan/vcpkg dependents` | that manager's recipe graph | opt-in; concretizer-exact but a **tiny fraction** of real use |
| Sourcegraph · `grep` · gh code-search | raw text matches | strings, **not identity**: no scoring, no fork/vendor dedupe, no graph |
| SBOM scanners (Syft, Trivy) | what a built **binary contains** | wrong direction, needs the artifact - your deps, not your dependents |

The common wall: **C++ has no registry and no universal manifest: the answer must come from the source.**

# Why I actually built this

🔬

### The scientists

Researchers I work with kept asking one simple thing: **is anyone actually using my library?** Nobody could give them an answer.

💥

### The broken builds

I kept making local changes that **broke Spack packages downstream** of the ones I maintained, with no way to see who I was about to break easily.

Interrogating the package manager made the first pass straightforward. **The rest of the problem was hard.**

# Three reasons this is worth the trouble



### Supply chain

When a small library has a vulnerability, **who is exposed?** You cannot patch, warn, or triage exposure you can't see.



### Impact & funding

"Who uses this?" is the question behind **every grant, every headcount, every 'is this still worth maintaining?'** Reach in code **and** in the literature that cites it.



### Blast radius

**Hyrum's Law**: with enough users, **every** observable behavior gets depended on. So who actually breaks when you change one? Dependents turn a guess into a checkable list.

All three are the same question: **who depends on me?**

DEPENDENCIES

# 2 A quick coverage of a solved problem

How do we find **dependencies**?

THE SHORT ANSWER

# You shouldn't have to

You should **know your dependencies**

...and yet, handed an unfamiliar project, we still have to go looking. So ... where?

# Where a project consumes its dependencies

- `manifests` `conanfile` · `vcpkg.json` · Spack env · `requirements`
- `containers` Docker images - scrape the layers
- `build system` `find_package` · linker lines · FetchContent / ExternalProject · pkg-config · CPS
- `VCS` git submodules
- `compile / link lines` the ground truth - but we rarely get to see them
- `C++ itself` how the code actually reaches for a library:
  - `#include` headers
  - `import` **modules**
  - `#pragma comment(lib, …)`
  - plugin architectures
  - namespaces (flakey, very hard to extract)

> The build system **feeds the compiler**; the code **names the library**. Both leave traces we can read.

# "Just interrogate the binary"

```
$ ldd ./app
  libssl.so.3  ⇒ ...
  libz.so.1    ⇒ ...
  libstdc++.so.6 ⇒ ...
```

```
$ nm -D ./app | c++filt
  deflate
  SSL_connect
  ...
```

- SO names and symbols are **generic**: `libz`, `deflate`, `SSL_*`.
- It needs the **built artifact**, per platform, per config.
- Maps a symbol back to **a** provider, not **your** provider.

DEPENDENTS

3

# Now the relfection

Dependencies → **dependents.** This is where it gets weird.

# The reflection




Flip a dependency graph and suddenly there's no way to obtain this information

> Searching for consumers of `foo` means

# Needle meet haystack.

Less a complex engineering problem, more **an exercise in thoroughness**

# Can we shrink the haystack?

There's too much hay, we need a smaller haystack... or maybe a magnet

**Five potential approaches, and a wall.**

Each buys something. None of them buys completeness.

# Magnet 1 - Package managers

SHRINK TO WHAT'S PACKAGED

**The idea**: enumerate consumers straight from the managers' own recipe graphs. A finite, closed set you can just walk.

```
spack dependents zfp
conan list --graph …
vcpkg depend-info zfp
```

**The catch**

- Opt-in - someone had to **write the recipe**
- Spack · Conan · vcpkg disagree, rename (`py-…`), split
- **Most C++ is never packaged - anywhere**

**Search space** · all public C++ — `finite - but a sliver`



packaged software — everything unpackaged

# Magnet 2 – Build systems

SHRINK TO DECLARED BUILDS

**The idea**: only look at repos whose build actually **wires the dependency to the linker**. That's specific, greppable syntax.

```
find_package(ZFP)          # CMake
bazel_dep(name = "zfp")    # Bazel
dependency('zfp')          # Meson
```

**The catch**

- You still have to **scan every repo** to find these files
- A dozen build systems, each with its own dialect
- **Vendored** and **header-only** consumers declare nothing



# Magnet 3 - Package config files

SHRINK TO THE CONFIG LAYER · CPS · PKG-CONFIG · CMAKE CONFIG

**The idea**: the installed-package **description** layer: machine-readable, build-tool-agnostic identity, designed to be parsed.

```
libzfp.pc          # pkg-config / pkgconf
zfpConfig.cmake    # CMake config-mode
zfp.cps            # CPS - the emerging cross-tool standard
```

**The catch**

- Describes **providers**, not who consumes them
- **CPS is brand-new**: minimal adoption yet; pkg-config is Unix-centric
- `FetchContent` / submodule users emit **none of it**
- None of them solve needing to read every repo

**Search space** · all public C++ — precise - but sparse



standard-config consumers — new standard · thin adoption

# Magnet 4 – Everything else

THE LONG TAIL OF CLEVER IDEAS

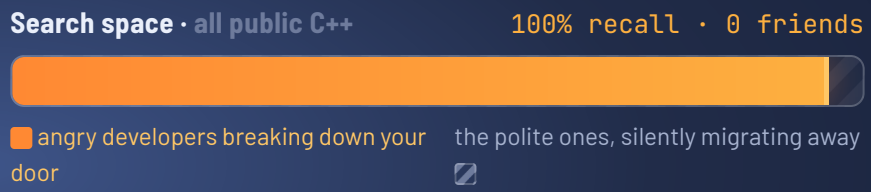
- **Docker image layers**: only what ships in a runtime
- **Published SBOMs · GitHub dependency graph**: only if someone uploaded one
- **GitHub code-search API**: rate-limited, shallow, GitHub-only
- **Binary / symbol indexes**: need the built artifact; names are generic
- **Downloads · telemetry**: executables only, and only if instrumented



# Magnet 5 – O(1)

Push a major breaking change with no warning at 2:58 AM on a random Tuesday.

**Wait.**




Every filter leaks. So we stop magnet fishing the haystack … and search all of it, precisely. →

PART FOUR

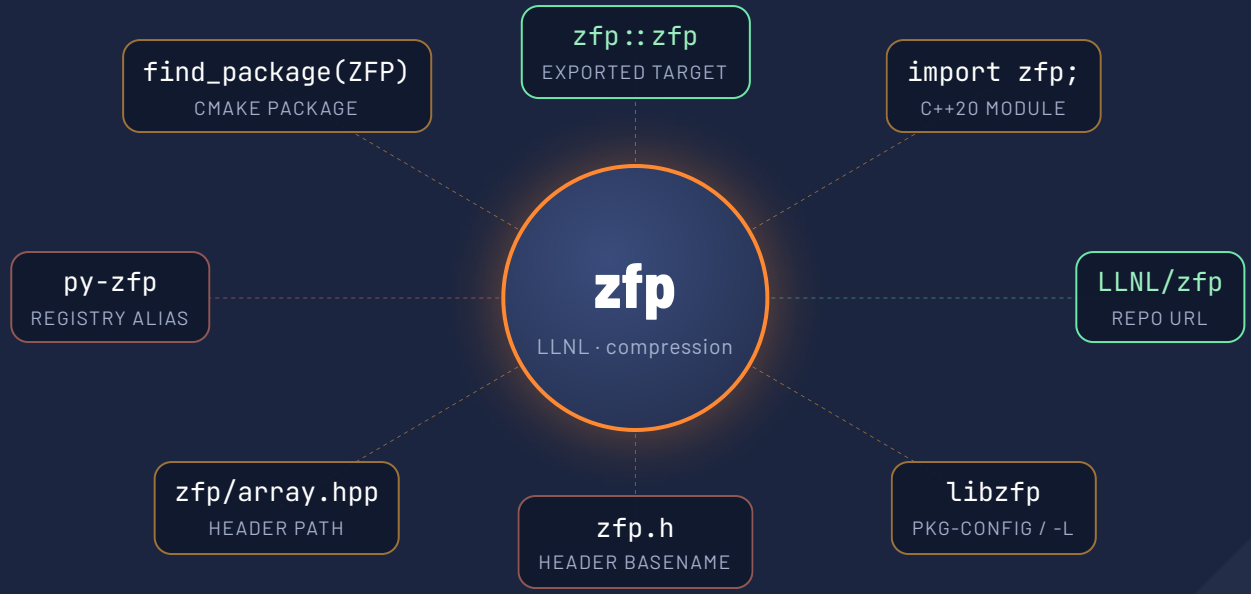


# Searching the haystack

Using the structure of C++ to define the search.

# Observe: the identifier set

One library does **not** have one name. It exposes a **bag** of loosely-related identifiers - and consumers reach for different ones.




So we **read the provider's own files**, learn every identifier, and record each with a kind, a

# C++ structure is our search plan

Each way the **provider** publishes itself maps to the exact line a **consumer** writes to reach for it.

| | PROVIDER SHIPS | | CONSUMER WRITES | SIGNAL |
|---|---|---|---|---|
| **Headers**<br>SOURCE | `zfp/array.hpp`<br>ships the header<br><br>bare basename #include <zfp.h> drops to low | → | `#include <zfp/array.hpp>` | HIGH |
| **C++20 modules**<br>SOURCE | `export module zfp;`<br>declares a module<br><br>no #include at all, so an include-only search misses it | → | `import zfp;` | MED |
| **CMake**<br>BUILD | `install(EXPORT zfpTargets NAMESPACE zfp::)`<br>exports a target | → | `find_package(ZFP)`<br>`target_link_libraries(app zfp::zfp)` | HIGH |
| **pkg-config**<br>BUILD | `libzfp.pc`<br>installs a .pc | → | `pkg_check_modules(ZFP libzfp)`<br>`-lzfp` | MED |
| **Repository**<br>VCS | `github.com/LLNL/zfp`<br>is the source<br><br>the one unambiguous identifier | → | `.gitmodules`<br>`FetchContent(GIT_REPOSITORY …)` | HIGH |

One combined regex, one **streaming** search over the global index, then classify every hit back to its kind.

# We don’t clone the world, we query it

The whole approach rests on a **high-availability index of public source**. Today that backend is **Sourcegraph**.

## What Sourcegraph gives us

- A continuously updated index of **millions of public repos**, already cloned and parsed.
- **Regex search** across all of it in a single query.
- A **streaming API** (SSE): hits arrive as they are found, no pagination dance.
- `fork:no`, repo metadata, and file paths: enough to dedupe and attribute a match.

## Why it carries the method

- The "brute force" only works because **someone already crawled the corpus**.
- One combined regex per provider becomes **one bounded query**, not a fleet of clones.
- The same endpoint powers the **IDF probes**: rarity is just a capped match count.
- It is one adapter behind our interface, so an **internal index drops in** the same way.

No private corpus of our own: we rent the haystack and bring the magnet.

# Where the structure thins out, heuristics take over

### High context → strong signal

- `find_package(ZFP)`: the call names the intent.
- `zfp::zfp`: the `::` is specific to how **this** library publishes.
- A `GIT_REPOSITORY` URL: **unambiguous.** One true identifier.

### Low context → we're guessing

- A bare `#include <config.h>`: whose config?
- A lone basename that collides with half of GitHub.
- **Namespaces in code**: sometimes a tell, usually noise, very hard to attribute.

> The strata where C++ gives the compiler **less** context are exactly where our search devolves into heuristics - so we treat them as weak, not wrong.

# A wide net catches noise, so we score, we don't discard

### ① IDF specificity gating

Drop tokens too common to attribute - but **only low-context ones** (bare basenames, project name). Never gate `find_package`, targets, or URLs: a popular `find_package(GTest)` is **real** dependents, not noise.

### ② Corroboration

```
score = strongest +
1.5·(layers-1) +
min(log10 hits, 2)
```

Bonus per **independent evidence layer** (source · build · registry · binary), not per repeated hit. high ≥ 5.5, med ≥ 3.0. An `#include` **and** a `find_package` **and** a `::` target → high; one lone basename → low. Prose-only is forced low.

### ③ Relationship, not deletion

A copy is not a user. Label edges `DEPENDS_ON` / `VENDORED` / `MIRROR`. **Keep and label - never drop.**

# The pipeline, end to end

01
**Compile identifier set**
read the provider's own files

02
**Gate for specificity**
IDF drops tokens too common to mean anything

03
**Expand to patterns**
each id → the line a consumer writes

04
**Stream-search the corpus**
one regex, SSE over Sourcegraph

05
**Classify each match**
which identifier? which evidence kind?

06
**Score & label**
tier + DEPENDS_ON / VENDORED / MIRROR

07
**Corroborate**
registries · GitHub metadata · citations

08
**Emit graph + SPDX**
nodes, edges, SBOM, completeness verdict

**Observe**
read provider files → identifiers

**Search**
one regex, streamed over the corpus

**Emit**
graph + SPDX + honesty report

# A partial answer must never look like a complete one

- Two people, same audit, **same graph**: except where live data changed.
- Every dropped request, truncated search, and hit cap is a **structured diagnostic.**
- Bounded steps are **deterministic**: rank **then** truncate, in a fixed order.

```
"completeness": {
  "complete": false,
  "warnings": [
    "discovery: results truncated
       for provider 'boost'",
    "citations: 3 nodes incomplete
       (rate_limited)"
  ]
}
```

A rate-limited crawl is reported as INCOMPLETE - never disguised as "no more results."

PART FIVE

# 5 Beyond the code

Impact you can't see in a build graph.

# Measuring scholarly impact, carefully

- **Seminal DOI**: the project's **own** paper (JOSS, `CITATION.cff`, Zenodo).
- **Citing DOIs**: a bounded reverse-citation crawl: **who cites the paper?**
- Every DOI is **relevance-scored**, not blindly attached.

### The colliding-name trap

`zfp` is also a term in an unrelated science. A bare keyword hit **can't** clear low on its own - it needs corroboration: shared **authors**, **venue**, **concepts**, terms.

How a paper was found is the strongest signal: a reverse-citation of your own paper » a bare keyword match.

# What comes out

### Dependency graph

One JSON of **nodes** + **edges**. Every edge carries its evidence, identifiers, provenance, confidence tier, and relationship. Sort, filter, drill in.

### SPDX 2.3 SBOMs

One SBOM snippet per edge, pinned to exact commit SHAs. Ingestible by compliance and security tooling - publishable to GitHub's "used by."

### Completeness verdict

One `complete` flag + sorted warnings. The run tells you how much of the truth it actually saw.

...and a dashboard renders it all. Here it is on a real library. ↓

# The output, live

corsa.center/dashboard/explore/dependents

## zfp ⓘ

Compressed numerical arrays that support high-speed random access

**507** dependents **459** organizations

**Dependents List** | Network Graph | Citations

| PACKAGE | OWNER | TYPE | STARS |
|---|---|---|---|
| godot | godotengine | L1 | ★ 107,283 |
| vcpkg Leaf | microsoft | L2 | ★ 26,696 |
| filament | google | L1 | ★ 19,728 |
| blender Leaf | blender | L2 | ★ 17,634 |
| bgfx Leaf | bkaradzic | L2 | ★ 16,806 |
| Open3D Leaf | isl-org | L2 | ★ 13,362 |

Showing 6 of 507 dependents · click any row → SPDX 2.3 snippet

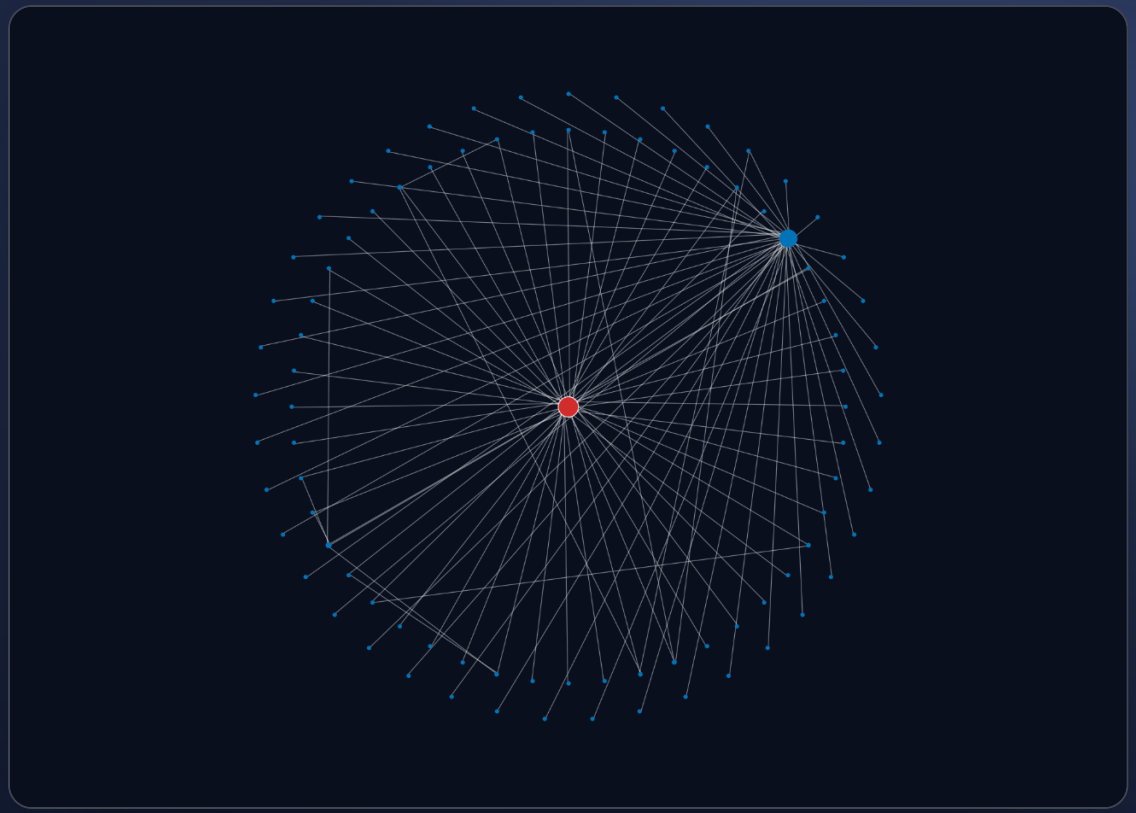
`zfp` - a compression library most people have never heard of - sits under a remarkable amount of software.

**507** dependents · **459** orgs · **69** papers

**Google**, **Microsoft**, **Godot**, **Blender**, **Open3D** - none of which any registry would have told `zfp` about.

Leaf = an end-user app · click any node → its **SPDX 2.3** snippet, pinned to a commit.

# The graph behind the list



Same data, drawn as the inverted tree the crawl actually walks.

- **zfp** - the root you searched for
- each spoke = a **real dependent** repository
- The far cluster is a dependent that is **itself** a hub - the crawl went **a level deeper** (depth 2)
- Exported straight from the dashboard - this is the actual output.

# Three ways to run it

### ① CLI

```
python audit_dependents.py \
  --repo LLNL/zfp --name zfp \
  --depth 1 --out graph.json \
  --sg-token … --gh-token …
```

One file, one dependency (`requests`). Plain HTTP - no clone, no DB.

### ② GitHub Action

```
- uses: johnwparent/dependent-auc
  with:
    root_repo: LLNL/zfp
    project_name: zfp
    max_depth: 1
    upload_artifact: true
```

Drop it in CI. Emits the graph + SPDX as build artifacts on every release.

### ③ Microservice

```
POST /audit          → { job_id }
GET  /jobs/{id}      → status
GET  /jobs/{id}/graph
GET  /jobs/{id}/spdx
```

Flask + worker in a Docker image. Queue a job, poll it, pull the graph - this is what feeds the dashboard.

Same engine underneath: a script, a CI step, or a service.

6

PART SIX

# So, what is this thing?

# What the tool actually is

🔌

### A modular interface to a source scraper

The C++ plugin is one implementation. **Roll your own backend** for a private corpus (say, your **50k internal repos**) so the dev you've never met doesn't walk to your desk because you broke payroll.

📚

### An optional publications miner

Attach the scholarly record when it exists: impact in **code and citations**.

**It builds a tree by BFS, but the tree is conceptually inverted.**

We find dependents. Then dependents of dependents.

# Honest limits

- **Public code only**: no private/internal usage without credentials.
- **Two backends wired up today**: Sourcegraph for the search, GitHub for metadata. The source interface is pluggable, so pointing it at your own internal index or forge is a small adapter, not a rewrite.
- **Intent, not linkage**: a match shows **intent to use**, not a shipped binary.
- **Bounded by the index**: very popular providers can exceed result limits (...and we say so).
- **Thin in, thin out**: unconventional layouts yield fewer identifiers.
- **Confidence, not ground truth**: high-confidence edges are very likely real; low ones are **leads for a human.**

> The design goal is graceful degradation: no single weak or missing signal is ever decisive.

TAKEAWAY

# Understand your dependents and understand your ecosystem impact.

Finding who `#includes` you is a search - so search the world's code, aim it with the structure of C++, and **score what you can't be sure of.**

GET IT

# It's open source.

**Run it on your own library.**

`github.com/corsa-center/dependent-audit`

A GitHub Action, a CLI, and a pluggable source interface.

# Thank you.

Who Is `#include`-ing You?

Measuring the Impact of C++ Libraries

**John Parent** · Kitware
`john.parent@kitware.com`

Questions welcome - beware of answers

